NaSCon '26

App Dev Competition

Part I: Multi-Sport League & Club Operations Platform

Problem Specification

Active Window	Day 1 Sprint (Saturday, Oct 17: 10:45 AM – 03:00 PM PKT)
Total Points	50 points (Backend Platform + Offline Mobile + Mobile UI/HCI — see Section 9)
Prizes	Winner: PKR 35,000 — Runner-Up: PKR 15,000
Venues	A-01, A-02, A-03, A-101, A-102
Final Push Deadline	15:05:00 PKT (Phase I is permanently locked at 15:00 PKT)

This document contains the complete, binding API contract. If any verbal statement conflicts with this document, this document wins.

Contents

1 Project Overview **1**

2 Phase I Engineering Outcomes **2**

3 Functional Requirements **2**

4 Non-Functional Requirements **4**

5 Mandatory Autograder Interface (`/audit/v1/*`) **4**

6 Hidden Complexity Edge Cases **18**

7 Approved Open-Source Stack **19**

8 Mobile Application Requirements and Judging **19**

9 Phase I Scoring and Rubric (50 Points) **20**

A Reference Code and Test Vectors **21**

B Error Code Catalogue **22**

C Pre-Submission Checklist **22**

1. Project Overview

Design and build a production-style, multi-tenant sports league and club-operations platform that combines rule-driven league management, fixture scheduling, club roster operations, and a concurrency-safe in-app merchandise store. Phase I has two deliverable surfaces:

1. **Mobile application** (Android or iOS — one platform suffices for 100% mobile credit) for players, club managers, league organizers, and signed-in viewers.
2. **Backend platform**: REST API, realtime WebSocket/SSE layer, event-sourced scoring engine, internal job queue, and admin configuration endpoints.

1.1 Open-Source and Self-Hosted Requirement

All core infrastructure must run on open-source software that the team self-hosts via Docker Compose. Proprietary single-vendor BaaS (Firebase, Supabase, AWS Amplify) for graded features is strictly prohibited. Payments must use a mock payment gateway that you build. Permitted components are listed in Section 7.

1.2 Phase I Hard Lock and Per-Team Cryptographic Salt

Part I (50 points) is evaluated automatically by Python grading scripts and is permanently locked at 15:00 PKT today (final GitHub push deadline: 15:05:00 PKT).

- Every event-sourced score record and checkout receipt must include a signature computed as `signature = HMAC_SHA256(DAY1_SALT, id + "|" + payload)`, using the unique `DAY1_SALT` printed on your team's slip.
- The exact byte-level recipe (meaning of `id` and `payload`, canonical JSON, encoding, output format) is in Section 5.3 and Appendix A. Follow it literally.
- The scope of Part II (Day 2: 45 objective points + 15 viva points) is confidential and will be released tomorrow at 10:15 AM PKT.

1.3 Submission Package and GitHub Rules

Work is graded from your GitHub repository. It must be **public** (the grader clones it without credentials) and its URL must be registered with the volunteer at your venue desk **before 11:15 AM**. The repository root must contain:

Path (repo root)	Requirement
`docker-compose.yml`	Brings up the entire backend (API on host port 8080, database, cache, queue, object store, workers). Must forward the salt as `DAY1_SALT: ${DAY1_SALT}`.
`.env.example`	Lists every environment variable. Must not contain your real salt.
`README.md`	Run instructions and the five demo accounts of Section 8.3.
`TRACEABILITY.md`	Maps every FR/NFR ID to source files and automated tests.
`tests/`, `run_tests.sh`, `COVERAGE.json`	Unit tests; one script that runs them; coverage file in the format of Section 9.3.
`mobile/`	Mobile source code and the built app at `mobile/release/app-release.apk` (Android). iOS teams: see Section 8.1.

GitHub push checkpoints. At least one commit must be pushed (GitHub push time) inside each of these windows: **10:45–11:45, 11:45–12:45, 12:45–13:45, and 13:45–14:45 PKT**. Each missed window costs 2 points (Section 9.4). The grader checks out the last commit pushed at or before 15:05:00 PKT; anything pushed later is ignored.

2. Phase I Engineering Outcomes

By completing Part I, teams demonstrate the ability to:

- model a flexible, rule-driven domain (multi-sport, multi-format) in a relational schema without hard-coded sport logic;
- implement and unit-test scheduling and standings algorithms (round-robin, Swiss, single and double elimination);
- build an append-only event-sourced scoring engine with compensating rollback events;
- implement 5-tier Role-Based Access Control (RBAC) with strict tenant isolation;
- deliver offline-tolerant mobile persistence with local action queueing and conflict synchronization;
- deliver a usable, accessible mobile interface that follows basic HCI and UI principles.

3. Functional Requirements

Each requirement carries an ID for traceability; maintain `TRACEABILITY.md` mapping every ID to source files and automated tests.

3.1 Sport and League Hierarchy Management

FR-1.1 Multi-Sport Taxonomy and Event-Sourced Scoring

- Support arbitrary sports (seed Football, Basketball, Cricket, Esports) *without code changes*. Adding a "mystery sport" during grading must work purely through JSON configuration (`POST /audit/v1/sports`).
- Each sport defines its rule set and scoring template *as data*: periods or innings structure, period duration, points per scoring event type, tie-break order, draw allowed (yes/no), overtime or shootout policy, roster size limits, and required positions.
- Scoring templates support three models: **event-based** (goals, baskets worth 1/2/3 points), **innings-based** (runs, wickets, overs), and **round/map-based** (esports best-of-N).
- **Event sourcing:** live match scores are computed from an append-only stream of match events. Corrections (e.g., a disallowed goal) are made by appending a compensating event signed with `DAY1_SALT`, never by mutating historical rows.

FR-1.2 League and Tournament Setup

Admins configure single-stage leagues, multi-division structures with promotion and relegation rules, knockout tournaments, and hybrid formats (group stage followed by a seeded knockout bracket), all scoped to seasons.

FR-1.3 Match and Fixture Scheduling

Custom implementation (third-party fixture-generation libraries are prohibited) supporting four formats:

1. **Round-robin:** single and double, correct home/away balance using the circle method, odd team counts handled via byes.
2. **Swiss-system:** pairing by score group, no repeat pairings, bye handling, configurable number of rounds, Buchholz tie-break.
3. **Single elimination:** byes for non-power-of-two fields and seeded bracket placement.
4. **Double elimination:** winners bracket, losers bracket, and grand final with optional bracket reset.

Also required:

- Manual rescheduling tools with real-time conflict detection: venue double-booking, team overlap, player overlap across squads (a player registered in two squads), minimum rest period between matches, and venue availability windows. Conflicts are returned as structured JSON errors identifying the conflicting entity IDs.
- Match status state machine: Upcoming, In-Progress, Completed, Postponed, Abandoned, Cancelled. Illegal transitions are rejected server-side. Status changes broadcast in real time to connected clients. Postponing a match automatically re-evaluates downstream bracket slots and re-runs conflict detection.

FR-1.4 Automated Standings and Tables

- Auto-generated points tables with configurable points per win/draw/loss.
- Tie-breakers in configurable order: (1) head-to-head points, (2) head-to-head differential (including 3-way mini-league), (3) overall differential, (4) goals/points scored, (5) fair-play points, (6) seeded drawing of lots.
- Head-to-head matrices for any pair of teams, sport-specific differentials (goal, point, run-rate, map), and playoff qualification status (clinched, eliminated, still possible), recomputed from scratch whenever a result or compensating event is recorded.

3.2 Club, Roster and Merchandise Store Operations

FR-2.1 Club Profiles and Tenant Isolation

Dedicated club portal with home venue coordinates, social handles, historical statistics across seasons, active roster, and staff (coach, manager, physio). Clubs belong to a white-label tenant (logo, colors, name) with strict tenant data isolation.

FR-2.2 Roster and Player Registry

- Add, transfer, loan (with start and end dates and automatic return), and release players across squads.
- Enforce configurable transfer windows per league, maximum squad size, one active registration per league season (except valid loans), minimum sport-specific position requirements, and role assignments (Captain, Vice-Captain, Starter, Substitute).
- Full immutable audit trail: every roster change records who, when, and why, viewable in history.

FR-2.3 Integrated Club Merchandise Store

- Product catalogue: apparel, accessories, and digital items with variants (size, color), scheduled price changes, and discount codes.
- **Concurrency-safe inventory (NFR-3):** stock tracked per variant, reserved during checkout against your mock payment gateway (authorization, capture, failure, refund simulation), guaranteeing zero overselling under concurrent purchases.
- Order states: Pending, Paid, Packed, Shipped, Delivered, Cancelled, Refunded; each transition emits a structured domain event to the internal queue.
- Server-side PDF receipt generation and embeddable product showcases on match pages (pinning a product updates connected clients in real time).

3.3 Cross-Cutting Access Control

FR-5.1 Five-Tier Role-Based Access Control

- Five tiers: Platform Super-Admin, League Organizer, Club Manager, Player, General Viewer.
- Scoped permissions: a League Organizer controls only their leagues; a Club Manager controls only their club; tenant isolation is absolute.
- Implement RBAC via a database policy matrix or policy-as-code layer, with negative authorization tests for every role boundary.

4. Non-Functional Requirements

ID	Requirement	Minimum acceptance bar
NFR-3	Concurrency correctness	Last 1 unit of stock bought by 20 simultaneous buyers yields exactly 1 successful order and 0 negative inventory (Section 5.12).
NFR-4	Offline tolerance	Mobile app caches fixtures, standings, and profiles locally (SQLite/-Room/Hive); offline scoring and roster actions queue locally and sync with conflict handling (Sections 5.14 and 8.4).
NFR-5	Security	OWASP Mobile/API Top 10 compliance; secure token storage (Keystore/K-eychain); rate limiting; input validation; 5-role RBAC enforcement (Section 5.13).
NFR-7	Reproducibility	One command (`docker compose up -d --build`) brings up the full back-end and seeds baseline data on a clean Linux machine (Section 5.2).

5. Mandatory Autograder Interface (`/audit/v1/*`)

The Python grading scripts evaluate your backend by calling the wrapper endpoints below on `http://localhost:8080`. Map these routes directly to your underlying domain services. Every request and response shape in this section is binding: field names, types, status codes, and error codes are compared literally by the scripts.

5.1 Endpoint Index

Method & path	Purpose	Sec.
`GET /health`	Readiness probe (no auth)	5.4
`POST /audit/v1/reset`	Wipe state and seed baseline data	5.5
`POST /audit/v1/sports`	Create a "mystery sport" from JSON rules	5.6
`POST /audit/v1/fixtures`	Generate round_robin / swiss / single_elim / double_elim	5.7
`POST /audit/v1/leagues`	Register a league (teams, fixtures, points, tie-break config)	5.8
`POST /audit/v1/events`	Append a match event or compensating event (signed)	5.9
`GET /audit/v1/score/{match_id}`	Score projection rebuilt from the event log	5.9
`GET /audit/v1/standings/{league_id}`	Ordered table, H2H, clinched/eliminated	5.10
`POST /audit/v1/roster/setup`	Create a roster scenario (league, squads, players)	5.11
`POST /audit/v1/roster/transfer`	Transfer / loan / release with all rule checks	5.11
`GET /audit/v1/roster/history/{player_id}`	Immutable audit trail + loan auto-return	5.11
`POST /audit/v1/checkout`	Concurrency-safe purchase + signed PDF receipt	5.12
`GET /audit/v1/inventory/{variant_id}`	Current stock of a variant	5.12
`POST /audit/v1/rbac-matrix`	JWTs for all 5 roles + scenario ids	5.13
`/audit/v1/rbac/*` (7 probe routes)	Authorization probes used with those JWTs	5.13
`GET /api/v1/ping`	Rate-limit probe	5.13
`POST /audit/v1/offline-sync`	Replay queued offline actions, return conflict resolutions	5.14

5.2 Global Conventions and Grading Environment

Grading environment and startup

- The grader runs on an organizer-controlled machine: Ubuntu 22.04, 4 vCPU, 8 GB RAM, Docker Engine 24+ with Compose v2, Python 3.11. Your stack must run within these limits.
- The grader clones your last commit (Section 1.3), then runs from the repository root: `DAY1_SALT=<your-salt> docker compose up -d --build`. The command (including image pulls and builds) must finish within **10 minutes**. Internet is available during this build step only; running services must not need it.
- The grader polls `GET /health` every 2 s; it must return HTTP 200 within **120 seconds** after the command returns. It then calls `POST /audit/v1/reset` before each test group.
- No manual steps, no pre-created volumes, no host-installed software other than Docker. Failure to start = non-reproducible stack (penalty in Section 9.4) after one automatic retry.
- `DAY1_SALT` must be read only from the environment variable at startup (never hard-coded or committed).

Request / response rules

Topic	Rule
Base URL / port	`http://localhost:8080`. All bodies are JSON, UTF-8, `Content-Type: application/json`.
Authentication	`/audit/v1/*` routes need no `Authorization` header, except the RBAC probe routes (5.13), which require `Authorization: Bearer <jwt>`.
Identifiers	Strings, 1–64 chars from `A-Za-z0-9_.:-`. Never contain `|`. Case-sensitive. Grader-chosen ids must be stored and returned verbatim.
Timestamps / dates	Timestamps: ISO 8601 UTC, `YYYY-MM-DDTHH:MM:SSZ`. Dates: `YYYY-MM-DD`.
Money	Integers in minor units (paisa). Currency is always `"PKR"`. 4500.00 PKR = `450000`.
Numbers	Integers where stated; otherwise JSON numbers. Net run rate is rounded to 3 decimals.
Unknown fields	Ignored. Missing required fields or wrong types → 400.
Validation order	1) malformed JSON/schema → 400; 2) referenced id does not exist → 404; 3) business rule violated → 409 (or 402 for payment). The first failing step wins.
Idempotency	Where a request carries a client id (`event_id`, `order_id`, `request_id`, `action_id`), replaying the identical request returns the stored result (HTTP 200) and changes nothing.
Rate limiting	`/audit/v1/*` and `/health` are exempt (the grader may send 200 req/s). `/api/v1/*` is limited to 100 requests per rolling 10 s per client IP (5.13).
Performance	Every request must finish within 5 s (reset within 10 s). The 20-request checkout burst must fully finish within 10 s.
Body size	Bodies larger than 1 MB are rejected with 413 `PAYLOAD_TOO_LARGE`.

HTTP status codes used

Code	Meaning in this contract
200	Success for reads, and for an idempotent replay of an earlier success.
201	A new resource/record was created or an action was applied for the first time.
400	Malformed JSON, schema violation, bad value (`VALIDATION_ERROR` and other 400 codes).
401	Missing, malformed, tampered, expired, or `alg:none` token.
402	Mock payment declined / capture failed (checkout only).
403	Authenticated but not allowed (role or tenant/ownership boundary).
404	A referenced id does not exist.
409	Business-rule or state conflict (illegal transition, out of stock, squad full, ...).
413	Body too large.
429	Rate limit exceeded (`/api/v1/*` only).

Error body (all non-2xx responses)

```
{
  "error": {
    "code": "OUT_OF_STOCK",
    "message": "Variant var-jersey-001-M has no stock left.",
    "details": { "variant_id": "var-jersey-001-M" }
  }
}
```

`code` is an UPPER_SNAKE_CASE string from the catalogue in Appendix B and is compared literally. `message` is free text. `details` is always an object (may be `{}`). For `VALIDATION_ERROR`, `details.fields` is an array of the offending field names.

5.3 Signature Specification (HMAC) — Exact Recipe

Used for event records (`/events`, and each event inside `/score`) and checkout receipts (`/checkout`).

Element	Definition
Key	The value of env `DAY1_SALT`, encoded as UTF-8 bytes (not hex-decoded, not trimmed).
`id`	Event: the `event_id`. Receipt: the `order_id`.
`payload`	The canonical JSON string of the record's payload object (events: the request's `payload`; receipts: `receipt.payload`).
Canonical JSON	Recursively sorted object keys (ascending by Unicode code point); no whitespace (separators `,` and `:`); UTF-8; non-ASCII characters *not* escaped; integers without decimal point; `true`/`false`/`null` lowercase. Array order preserved.
Message	`id + "|" + payload` as UTF-8 bytes.
Algorithm / output	HMAC-SHA256; output = lowercase hex, 64 characters.

The grader recomputes every signature with its copy of your salt and compares for equality. A reference implementation is in Appendix A.

5.4 Health Probe

`GET /health`

Returns `200 {"status":"ok"}` only when the database is reachable and baseline data is seeded. No auth.

5.5 Reset and Baseline Data

`POST /audit/v1/reset`

Wipes all state (matches, events, leagues, rosters, orders, offline-sync records, custom sports) and re-creates the baseline below. Must be repeatable and leave identical state every time.

Request (body optional)

```
{ "seed": 42, "stock": { "var-jersey-001-M": 1, "var-jersey-001-L": 5 } }
```

- `seed`: integer, optional, default 42; used for the drawing-of-lots tie-break (5.10).
- `stock`: map `variant_id` → integer ≥ 0; defaults: M = 1, L = 5.

Response 200

```
{
  "status": "reset_complete",
  "seed": 42,
  "sports": ["football", "basketball", "cricket", "esports"],
  "tenants": ["tenant-alpha", "tenant-beta"],
  "stock": { "var-jersey-001-M": 1, "var-jersey-001-L": 5 }
}
```

Baseline data that `/reset` must create (exact ids)

Item	Seed content
Tenants	`tenant-alpha` ("Alpha Sports"), `tenant-beta` ("Beta Athletics").
Store	Product `prod-jersey-001` (tenant-alpha). Variants `var-jersey-001-M` and `var-jersey-001-L`, each `price_minor = 450000`, currency PKR. Discount code `WELCOME10` = 10% off subtotal.
RBAC scenario	Leagues `league-alpha-1`, `league-alpha-2` (tenant-alpha), `league-beta-1` (tenant-beta); clubs `club-alpha-1`, `club-alpha-2`, `club-beta-1`; players `player-alpha-1` (club-alpha-1), `player-alpha-2` (club-alpha-2), `player-beta-1` (club-beta-1).
Demo users	The five accounts of Section 8.3, bound to the scenario above.
Sports	The four sports below, stored as ordinary data using the same schema as 5.6.

sport_id	Model	Periods	scoring_events	Draw	Overtime	Roster
football	event	2×45 min	goal = 1 point	yes	none	min 11, max 25, gk ≥ 1
basketball	event	4×12 min	free_throw = 1, two_pointer = 2, three_pointer = 3	no	extra_time 5 min	min 5, max 15
cricket	innings	1 innings/team, 20 overs, 6 balls/over, 10 wickets	dot_ball (0 runs, legal), single (1), two (2), four (4), six (6), wide (1 run, not legal), no_ball (1 run, not legal), wicket (0 runs, legal, wicket)	yes	super_over	min 11, max 18, wicketkeeper ≥ 1
esports	round	best_of 3	round_win = 1 round	no	none	min 5, max 8

5.6 Mystery Sport from JSON

`POST /audit/v1/sports`

Creates a sport purely from data. The new sport must be immediately usable in `/events`, `/leagues`, `/standings`, and `/fixtures` without any code change or restart. Event names are arbitrary — never hard-code names such as "goal" or "wicket".

Example request (event model)

```
{
  "sport_id": "kabaddi",
  "name": "Kabaddi",
  "scoring_model": "event",
  "periods": { "count": 2, "duration_minutes": 20 },
  "scoring_events": {
    "raid_point": { "points": 1 },
    "tackle_point": { "points": 1 },
    "all_out": { "points": 2 }
  },
  "draw_allowed": false,
  "overtime": { "policy": "golden_score", "duration_minutes": 5 },
  "tie_break_order": ["h2h_points", "h2h_diff", "overall_diff", "scored", "fair_play", "draw_lots"],
  "roster": { "min_squad_size": 7, "max_squad_size": 12,
              "required_positions": { "raider": 2, "defender": 3 } }
}
```

Fragments for the other two models

```
innings model:
  "periods": { "innings_per_team": 1, "overs_per_innings": 10, "balls_per_over": 6, "wickets_per_innings": 10 },
  "scoring_events": { "super_six": { "runs": 6, "legal_ball": true, "wicket": false },
                      "bye_run":   { "runs": 1, "legal_ball": false, "wicket": false },
                      "out":       { "runs": 0, "legal_ball": true, "wicket": true } }
round model:
  "periods": { "best_of": 5 },
  "scoring_events": { "map_win": { "rounds": 1 } }
```

Field	Type	Rules
`sport_id`	string	Required. Regex `^[a-z0-9_-]{2,40}$`. Duplicate → 409 `SPORT_EXISTS`.
`name`	string	Required, 1–80 chars.
`scoring_model`	enum	Required: `event`, `innings`, or `round`.
`periods`	object	Required. *event:* `count` ≥ 1, `duration_minutes` ≥ 1. *innings:* `innings_per_team`, `overs_per_innings`, `balls_per_over`, `wickets_per_innings` (integers ≥ 1). *round:* `best_of` (odd integer ≥ 1).
`scoring_events`	object	Required, ≥ 1 key; key regex `^[a-z0-9_]{1,40}$`. Value per model — *event:* `{points}` (int ≥ 1); *innings:* `runs` (int ≥ 0), `legal_ball` (bool), `wicket` (bool); *round:* `{rounds}` (int ≥ 1).
`draw_allowed`	boolean	Required.
`overtime`	object	Required: `policy` is one of `none`, `extra_time`, `shootout`, `golden_score`, `super_over`; optional `duration_minutes` (int ≥ 0).
`tie_break_order`	array	Optional. Items from `h2h_points`, `h2h_diff`, `overall_diff`, `scored`, `fair_play`, `draw_lots` (no repeats). Default = the full list. `draw_lots` is always applied last, even if omitted.
`roster`	object	Required: `min_squad_size`, `max_squad_size` (ints ≥ 1, min ≤ max), `required_positions` (position → minimum count, may be `{}`).

Responses: `201 { "sport_id": "kabaddi", "status": "created", "config": {...stored config,`

`echoed...} }`; 400 `VALIDATION_ERROR` (`details.fields` lists bad fields); 409 `SPORT_EXISTS`.

5.7 Schedule and Bracket Generation

`POST /audit/v1/fixtures`

Pure computation: the endpoint stores nothing. Input is `{format, teams, params}`; `teams` is an array of unique team-id strings. For `swiss`, `single_elim`, and `double_elim` the array order is the seed order (index 0 = seed 1). Limits: round_robin 2–40 teams; swiss 2–64; single_elim 2–128; double_elim 3–128; violations → 400 `VALIDATION_ERROR`. The scripts check rules and structure, never one specific schedule (many valid schedules exist) — except where a section says "exact".

A) format = round_robin

```
{ "format": "round_robin", "teams": ["t1","t2","t3","t4","t5"],
  "params": { "legs": 1,
              "withdrawals": [ { "team_id": "t3", "after_round": 2 } ],
              "withdrawal_policy": "forfeit_remaining" } }
```

- `legs`: 1 (single) or 2 (double); default 1. `withdrawals`: default `[]`; `after_round` = the last (global) round the team still plays; 0 = never plays. `withdrawal_policy`: `void_all` or `forfeit_remaining` (default).

Response 200

```
{ "format": "round_robin", "legs": 1, "team_count": 5, "round_count": 5,
  "rounds": [
    { "round": 1,
      "matches": [ { "match_id": "RR-L1-R1-M1", "leg": 1, "home": "t2", "away": "t5",
                     "status": "scheduled", "awarded_to": null },
                   { "match_id": "RR-L1-R1-M2", "leg": 1, "home": "t3", "away": "t4",
                     "status": "scheduled", "awarded_to": null } ],
      "byes": ["t1"] } ],
  "summary": { "total": 10, "scheduled": 8, "forfeit": 2, "voided": 0 } }
```

- Rounds per leg: $n-1$ (n even) or n (n odd). Matches per round: $\lfloor n/2 \rfloor$. `byes` is `[]` for even n, exactly one team for odd n. Round numbers are global: leg 2 continues after leg 1. `match_id` format `RR-L{leg}-R{round}-M{index}` (index from 1 within the round).
- Each team plays exactly once per round, except its bye round. For odd n every team has exactly one bye per leg.
- Every unordered pair meets exactly `legs` times; with `legs = 2` every ordered pair (home, away) occurs exactly once.
- Home/away balance (circle method): within each leg, for every team $|\text{home games} - \text{away games}| \leq 1$.
- `forfeit_remaining`: fixtures involving a withdrawn team with `round > after_round` get `status:"forfeit"` and `awarded_to` = the opponent; earlier fixtures stay `scheduled`. `void_all`: every fixture involving the withdrawn team (any round) gets `status:"voided"`, `awarded_to:null`. A fixture where both teams are withdrawn is voided. Fixtures are never removed from the list. Byes are unaffected.
- `withdrawals` with an unknown team, or `after_round` outside `0..round_count` → 400.

B) format = swiss

One call pairs one round. The caller supplies the results of earlier rounds in `history`.

```
{ "format": "swiss", "teams": ["t1","t2","t3","t4","t5"],
  "params": { "total_rounds": 3, "round": 2,
    "history": [ { "round": 1, "bye": "t5",
                   "matches": [ {"home":"t1","away":"t3","result":"home"},
                                {"home":"t2","away":"t4","result":"draw"} ] } ] } }
```

- `total_rounds`: integer $1..n-1$; default $\lceil \log_2 n \rceil$. `round`: the round to pair, 1..total_rounds; default 1. `history` must contain exactly rounds 1..round − 1, else 400. `history[].matches[].result`: `home`, `away`, or `draw`. `history[].bye`: team id that received a bye, or `null`.

Response 200

```
{ "format": "swiss", "round": 2, "total_rounds": 3,
  "pairings": [ { "match_id": "SW-R2-M1", "home": "t1", "away": "t2" },
                { "match_id": "SW-R2-M2", "home": "t5", "away": "t4" } ],
  "bye": "t3",
  "standings": [ { "rank": 1, "team_id": "t1", "score": 1.0, "buchholz": 0.0 },
                 { "rank": 2, "team_id": "t5", "score": 1.0, "buchholz": 0.0 },
                 { "rank": 3, "team_id": "t2", "score": 0.5, "buchholz": 0.5 }, ... ] }
```

- **Scoring:** win = 1, draw = 0.5, loss = 0, bye = 1 (counted as a win).
- `standings` reflect history only (before the round being paired), sorted by score desc, then buchholz desc, then

seed (lower index first). Buchholz = sum of the current scores of every opponent the team actually played; a bye contributes 0. `score` and `buchholz` are compared exactly.

- **Round 1 (exact):** let $m = \lfloor n/2 \rfloor$. If n is odd the last seed gets the bye. Pair seed i with seed $i + m$ for $i = 1..m$ (home = lower seed number).
- **Rounds ≥ 2 (checked as properties):** (a) every team appears at most once; (b) no pair has met before; (c) with odd n exactly one bye, given to the lowest-scoring team that has not had a bye (tie → lowest seed, i.e. largest index); if no valid pairing exists for the rest, try the next-lowest candidate; (d) the sum over pairs of |score difference| equals the minimum achievable over all rematch-free pairings — greedy pairing that gets stuck will fail; use backtracking/matching. Home/away is not checked.
- If no complete rematch-free pairing exists for any bye candidate: 409 `NO_VALID_PAIRING` with `details.round`.

C) format = single_elim

```
{ "format": "single_elim", "teams": ["t1","t2","t3","t4","t5","t6"],
  "params": { "third_place_match": false, "results": [ { "match_id": "R1M2", "winner": "t4" } ] } }
```

Bracket construction (exact). Bracket size B = next power of two $\geq n$. Seed order is built recursively: order(2) = [1, 2]; order($2k$) = for each s in order(k): $[s,\ 2k + 1 - s]$. So order(8) = [1, 8, 4, 5, 2, 7, 3, 6]. Round-1 match i takes the $(2i - 1)$-th and $2i$-th entries; the lower seed number is home. A seed number $> n$ is missing: that match has `bye:true`, `away:null`, and its winner is the present team. Example $n = 6$, $B = 8$: R1M1: seed 1 vs (seed 8 missing) BYE; R1M2: seed 4 vs seed 5; R1M3: seed 2 vs (seed 7 missing) BYE; R1M4: seed 3 vs seed 6.

Response 200

```
{ "format": "single_elim", "bracket_size": 8,
  "rounds": [ { "round": 1, "matches": [ MATCH, MATCH, MATCH, MATCH ] },
              { "round": 2, ... }, { "round": 3, ... } ] }

MATCH = { "match_id": "R1M2", "bracket": "main", "round": 1, "index": 2,
          "home": "t4", "away": "t5", "home_seed": 4, "away_seed": 5, "bye": false,
          "winner": null, "loser": null,
          "winner_to": { "match_id": "R2M1", "slot": "away" }, "loser_to": null }
```

- `match_id = R{round}M{index}`; index counts from 1 in bracket order. Rounds = $\log_2 B$. Final: `winner_to:null`.
- Advancement: winner of `R{r}M{i}` goes to `R{r+1}M{ceil(i/2)}`, slot `home` if i is odd, `away` if even.
- `home`/`away` are `null` while unknown. Bye winners are propagated immediately (e.g. `R2M1.home = t1`).
- `params.results`: each item `{match_id, winner}`; the winner is filled into `winner`/`loser` and propagated into `winner_to`. Unknown `match_id`, or a winner who is not a participant of that match → 400.
- `third_place_match:true` adds a match `TP` (`bracket:"third_place"`) between the two semifinal losers (`loser_to` of both semifinals points to `TP`, slots `home`/`away`).
- The MATCH fields shown are mandatory (plus the conditional/required fields for double elimination).

D) format = double_elim

```
{ "format": "double_elim", "teams": ["t1","t2","t3","t4","t5","t6","t7","t8"],
  "params": { "bracket_reset": true, "results": [ { "match_id": "WR1M1", "winner": "t1" } ] } }
```

Response 200

```
{ "format": "double_elim", "bracket_size": 8, "bracket_reset": true,
  "winners": [ { "round": 1, "matches": [ MATCH... ] }, ... ],
  "losers":  [ { "round": 1, "matches": [ MATCH... ] }, ... ],
  "grand_final": [ GF1_MATCH, GF2_MATCH ] }
```

- B and the WR1 seeding are identical to single elimination (exact). Same MATCH object; `bracket` is one of `winners`, `losers`, `grand_final`.
- **Ids and counts (exact).** Winners bracket `WR{r}M{i}`: round r has $B/2^r$ matches ($r = 1..k$, $B = 2^k$). Losers bracket `LR{r}M{i}`: rounds $1..2(k-1)$; rounds $2j-1$ and $2j$ each have $B/2^{j+1}$ matches ($j = 1..k-1$). Total losers matches = $B - 2$. Grand final `GF1`; `GF2` exists only when `bracket_reset:true`.
- **Routing** (checked as a graph; exact slot mapping inside the losers bracket is free). Every non-terminal match has `winner_to`; every winners-bracket match has `loser_to`; losers-bracket matches have `loser_to:null`; every match slot except WR1 is fed by exactly one source. WR final winner → `GF1.home`; WR final loser → LB final `away`; LB final winner → `GF1.away`. Recommended routing: WR1 losers meet each other in LR1; WRr ($r \geq 2$) losers drop into LR($2r - 2$).
- **Bracket reset.** `GF2` has `conditional:true` and the same two teams as `GF1`. Its `required` is `null` until GF1 has a result; then `true` iff `bracket_reset` is true and the GF1 winner is the losers-bracket champion (`GF1.away`); otherwise `false`. A winners-bracket champion who wins GF1 ends the event.

- WR1 byes (non-power-of-two fields) are graded as in single elimination. Whether losers-bracket matches fed by a missing team carry `bye:true` is not graded.

5.8 Register a League for Standings

`POST /audit/v1/leagues`

Registers teams, the full fixture list, and standings configuration. Each fixture automatically becomes a match stream (status `upcoming`) fed through `/events`. Used by `/standings`.

```
{
  "league_id": "lg-1",
  "sport_id": "football",
  "teams": ["t1", "t2", "t3", "t4"],
  "fixtures": [ { "match_id": "m1", "home": "t1", "away": "t2" },
                { "match_id": "m2", "home": "t3", "away": "t4" } ],
  "points": { "win": 3, "draw": 1, "loss": 0 },
  "tie_break_order": ["h2h_points", "h2h_diff", "overall_diff", "scored", "fair_play", "draw_lots"],
  "qualification": { "spots": 2 },
  "lots_seed": 42
}
```

- `league_id`, `sport_id`, `teams`, `fixtures`: required. Sport must exist (404 `SPORT_NOT_FOUND`). Teams unique (2–40). Fixture teams must be in `teams`, home ≠ away. `match_id` already used → 409 `MATCH_EXISTS`; league id used → 409 `LEAGUE_EXISTS`.
- `points`: optional ints, default win 3 / draw 1 / loss 0. `tie_break_order`: optional, same tokens as 5.6; default = the sport's order.
- `qualification.spots`: required int 1..number of teams (top-N qualify for the playoffs).
- `lots_seed`: optional int; default = the seed given to the last `/reset` (42 if none).
- **Response 201:** `{"league_id":"lg-1","status":"created","team_count":4,"fixture_count":2}`.

5.9 Events and Score Projection

`POST /audit/v1/events`

```
{
  "event_id": "evt-0003",
  "match_id": "m1",
  "type": "score",
  "payload": { "team_id": "t1", "event_type": "goal" }
}
```

A match stream exists if it is a league fixture (5.8) or its `match_init` was accepted. `match_init` for an existing stream → 409 `MATCH_EXISTS`; any other event for a missing stream → 404 `MATCH_NOT_FOUND`.

type	payload	Allowed when / rule
`match_init`	`{sport_id, home_team_id, away_team_id}` (+ optional `league_id`)	Creates a stand-alone match, status `upcoming`. Unknown sport → 404 `SPORT_NOT_FOUND`.
`match_start`	`{}`	Status must be `upcoming` → becomes `in_progress`; else 409 `ILLEGAL_TRANSITION`.
`score`	`{team_id, event_type}`	Status `in_progress` (else 409 `MATCH_NOT_IN_PROGRESS`). `team_id` must be home or away (else 400). `event_type` must exist in the sport's `scoring_events` (else 400 `UNKNOWN_EVENT_TYPE`). Points/runs/rounds come from the sport config.
`discipline`	`{team_id, points}` (integer, usually negative)	Status `in_progress`. Adds to the team's fair-play total (higher is better).
`tiebreak_winner`	`{team_id}`	Status `in_progress`. Declares the winner of a shootout / overtime / super over without changing the score.
`match_end`	`{}`	Status must be `in_progress` (else 409 `ILLEGAL_TRANSITION`) → `completed`. If the score is level, `draw_allowed` is false, and no active `tiebreak_winner` exists → 409 `TIE_NOT_ALLOWED`.
`compensation`	`{compensates, reason}`	Any status after init, including `completed`. `compensates` = `event_id` of a `score`, `discipline`, or `tiebreak_winner` event of the same match. Unknown → 404 `EVENT_NOT_FOUND`; already compensated, other type, or other match → 409 `INVALID_COMPENSATION`.

- **Model limits (config-driven).** *innings model:* a team's legal balls ≥ `innings_per_team`×`overs_per_innings`×`balls_per_over`, or wickets ≥ `innings_per_team`×`wickets_per_innings` → further score events for that team are 409 `INNINGS_COMPLETE`. *round model:* once a team has ≥ `(best_of + 1) / 2` rounds, further score events → 409 `SERIES_ALREADY_DECIDED`. Compensated events do not count toward these limits.
- **Append-only log.** Compensation appends a new signed event; the original row is never modified. The projection treats a compensated event (and the compensation itself) as neutral.
- **Idempotency.** Same `event_id` with an identical body → 200 with the stored response. Same `event_id` with a different body → 409 `DUPLICATE_EVENT_ID`.
- The score must be rebuilt from the event log on read (no mutable counters as the source of truth).

Event response — 201 (200 on idempotent replay)

```
{ "event_id": "evt-0003", "match_id": "m1", "type": "score", "sequence": 3, "match_version": 3,
  "signature": "9f2c...64 lowercase hex chars..." }
```

`sequence` = 1-based position in that match's log (compensations and every other type count). `match_version` = number of events in the log after this append. `signature` per Section 5.3 with `id = event_id` and the canonical JSON of the request payload.

`GET /audit/v1/score/{match_id}`

Response 200

```
{
  "match_id": "m1", "sport_id": "football", "league_id": "lg-1",
  "status": "completed",
  "outcome": "home_win",
  "winner_team_id": "t1",
  "tiebreak_winner_team_id": null,
  "home": { "team_id": "t1", "score": 2, "wickets": null, "balls": null, "overs": null, "fair_play": 0 },
  "away": { "team_id": "t2", "score": 1, "wickets": null, "balls": null, "overs": null, "fair_play": -1 },
  "version": 9,
  "events": [ { "sequence": 1, "event_id": "evt-0001", "type": "match_init", "payload": {...},
                "signature": "...", "compensated": false, "compensated_by": null }, ... ]
}
```

Field	Definition
`status`	`upcoming`, `in_progress`, or `completed` (derived from the log).
`outcome`	`pending` while not completed; when completed: `home_win`, `away_win`, `draw`, or `undecided` (level score, `draw_allowed` false, no active tie-break winner — e.g. after the tie-break event was compensated).
`score`	event model: sum of points. innings model: total runs. round model: rounds won. Active (non-compensated) events only.
`wickets`, `balls`, `overs`	innings model only (else `null`). `balls` = legal balls faced by that team; `overs` = string `"{balls div bpo}.{balls mod bpo}"`, e.g. `"3.2"`.
`winner_team_id`	Higher score; if level, the active `tiebreak_winner` team; else `null`.
`events`	All events in sequence order, including compensated ones and compensations. `compensated` is true for a targeted event; `compensated_by` holds the compensating `event_id`.

5.10 Standings

`GET /audit/v1/standings/{league_id}`

Recomputed from scratch from the event logs on every call. Only matches with status `completed` and outcome $\neq$ `undecided` count as played; all other fixtures are "remaining".

```
{
  "league_id": "lg-1", "sport_id": "football", "qualification_spots": 2,
  "table": [
    { "rank": 1, "team_id": "t1", "played": 3, "won": 2, "drawn": 1, "lost": 0,
      "scored": 7, "conceded": 2, "diff": 5, "points": 7, "fair_play": 0,
      "status": "clinched", "resolved_by": null }, ...
  ],
  "h2h": { "t1": { "t2": { "played": 1, "points": 3, "scored": 2, "conceded": 1 }, ... }, ... }
}
```

Field	Definition
`points`	won × win + drawn × draw + lost × loss. A win decided by `tiebreak_winner` counts as a win.
`scored`/`conceded`	Sum of `score` for / against (goals, points, runs, rounds).
`diff` (event model)	scored − conceded.
`diff` (round model)	Rounds won − rounds lost (map differential).
`diff` (innings model)	Net run rate = total runs scored ÷ total overs faced − total runs conceded ÷ total overs bowled, rounded to 3 decimals. Overs = legal balls ÷ `balls_per_over` (decimal). An innings that ended with wickets ≥ `wickets_per_innings` counts as the full `overs_per_innings`. Overs bowled = the opponent's overs faced.
`fair_play`	Sum of the team's active `discipline` points.
`h2h`	`h2h[a][b]` = results of a against b over all completed matches between them: `played`, `points` earned by a, `scored` by a, `conceded` by a. Present for every ordered pair of teams (zeros when never met).
`resolved_by`	Name of the criterion that placed this team above the next team it was tied with on points (`h2h_points`, `h2h_diff`, `overall_diff`, `scored`, `fair_play`, `draw_lots`), or `null` if it was not tied.

Ordering algorithm (exact)

1. Sort teams by points descending. Teams with equal points form a tied group.
2. Resolve each tied group with `resolve(group)`: for each criterion in `tie_break_order` (then `draw_lots`), compute the criterion for every team of the group; if the values are not all equal, split the group into sub-groups by value (best first); every sub-group of size > 1 is resolved by calling `resolve` on it again *from the first criterion*; stop.
3. `h2h_points` = points earned only in completed matches among the teams of the current group (the 3-way mini-league when the group has three teams). `h2h_diff` = `diff` computed only over those same matches (net run rate over those matches for innings sports). `overall_diff`, `scored`, `fair_play` use all completed league matches. Higher is better for all.
4. `draw_lots`: order the remaining teams ascending by `SHA256_hex(lots_seed + "|" + league_id + "|" + team_id)` (decimal seed, UTF-8). First = best. Deterministic, so the grader compares it exactly.

Qualification status (exact; uses points only, tie-breaks ignored)

Let W = win points, $R(T)$ = number of remaining fixtures of team T, $\max(T) = \text{points}(T) + W \times R(T)$, and k = `qualification.spots`.

- `clinched` if fewer than k *other* teams X satisfy $\max(X) \geq \text{points}(T)$.
- `eliminated` if at least k other teams X satisfy $\text{points}(X) > \max(T)$.
- otherwise `still_possible`. (Clinched is checked first.)

5.11 Roster, Transfers, Loans and History

`POST /audit/v1/roster/setup`

Builds a scenario; re-sending the same `league_id` replaces that league's roster state.

```
{
  "league": { "league_id": "lg-r1", "tenant_id": "tenant-alpha", "sport_id": "football",
              "transfer_window": { "open": "2026-09-01", "close": "2026-09-30" },
              "max_squad_size": 3, "required_positions": { "gk": 1 } },
  "squads": [ { "squad_id": "sq-a", "club_id": "club-a", "tenant_id": "tenant-alpha", "name": "A" },
              { "squad_id": "sq-b", "club_id": "club-b", "tenant_id": "tenant-alpha", "name": "B" } ],
  "players": [ { "player_id": "p1", "name": "Ali", "position": "gk", "squad_id": "sq-a", "role": "captain" },
               { "player_id": "p2", "name": "Bilal", "position": "df", "squad_id": "sq-a", "role": "starter" } ],
  "matches_in_progress": [ { "match_id": "m-live", "home_squad_id": "sq-a", "away_squad_id": "sq-b" } ]
}
```

Response 201: `{"league_id":"lg-r1","status":"configured","squad_counts":{"sq-a":2,"sq-b":0}}`. `role` is one of `captain`, `vice_captain`, `starter`, `substitute`. `required_positions` = minimum players per position a squad must keep. If `max_squad_size`/`required_positions` are omitted they default from the sport's roster config. Window dates are inclusive. A player is "in an in-progress match" when their squad is the home or away squad of any entry in `matches_in_progress`.

`POST /audit/v1/roster/transfer`

```
{
  "request_id": "tr-001", "league_id": "lg-r1", "player_id": "p2",
  "type": "transfer",
  "from_squad_id": "sq-a", "to_squad_id": "sq-b",
  "effective_date": "2026-09-30",
  "loan_end_date": null,
  "reason": "Tactical move", "requested_by": "user-77"
}
```

`type` is `"transfer"`, `"loan"`, or `"release"`. `to_squad_id` is `null` for `release`. `loan_end_date` is required (and > `effective_date`) only for `loan`.

Rule checks — evaluated in this exact order; the first failure is returned

#	Check	Result
1	Schema, dates, `loan_end_date` > `effective_date`, `to_squad_id` present unless release	400 `VALIDATION_ERROR`
2	League / player / squads exist	404 `LEAGUE_NOT_FOUND`, `PLAYER_NOT_FOUND`, `SQUAD_NOT_FOUND`
3	From and to squads belong to the league's tenant	409 `CROSS_TENANT_TRANSFER`
4	Player's current squad in this league equals `from_squad_id`	409 `NOT_IN_SQUAD`
5	transfer/loan: `effective_date` inside the window (inclusive). Release is allowed on any date.	409 `TRANSFER_WINDOW_CLOSED`
6	Player is not currently on loan (transfer/loan/release of a loaned player is refused)	409 `PLAYER_ON_LOAN`
7	transfer/loan: destination squad count $+1 \leq$ `max_squad_size`	409 `SQUAD_FULL`
8	After the move, the from squad still has $\geq$ the required count for the player's position	409 `MIN_POSITION_VIOLATION`

- **Effects.** *transfer:* the player moves permanently, new role = `substitute`; if the old role was captain/vice_captain that slot in the old squad becomes vacant. *loan:* the player belongs to the borrower

from `effective_date` through `loan_end_date` (inclusive); `parent_squad_id` = the lender; the player counts toward the borrower's squad size and no longer toward the lender's. *release:* the player leaves the league (no squad).

- **Automatic return.** The day after `loan_end_date` the player is back with the lender (role `substitute`). Because the grader cannot wait, it passes `as_of` to the history call (below); the return must then appear as a `loan_return` audit entry (written once, never duplicated on repeated calls).
- **In-progress match (edge case 6).** If the player is in an in-progress match, the request is still accepted (the window is judged on `effective_date`, so the last window day works) with `state:"deferred_until_match_end"` and `blocking_match_id`; the player stays eligible for that match with the old squad. Otherwise `state:"active"`, `blocking_match_id:null`.
- **Audit trail.** Every accepted roster change appends one immutable entry. Rejected requests change nothing and write no entry. Idempotency by `request_id` (replay → 200, same body).

Transfer response — 201 (200 on replay)

```
{ "request_id": "tr-001", "status": "accepted", "state": "active", "blocking_match_id": null,
  "registration": { "player_id": "p2", "league_id": "lg-r1", "squad_id": "sq-b", "parent_squad_id": null,
                    "registration_type": "transfer", "start_date": "2026-09-30", "loan_end_date": null,
                    "role": "substitute" },
  "audit_entry": { "audit_id": "au-0001", "action": "transfer", "player_id": "p2", "league_id": "lg-r1",
                   "from_squad_id": "sq-a", "to_squad_id": "sq-b", "effective_date": "2026-09-30",
                   "state": "active", "who": "user-77", "when": "2026-09-30T08:12:44Z",
                   "why": "Tactical move" } }
```

For `release`, `registration` is `null`. For `loan`, `registration_type:"loan"`, `parent_squad_id` = lender, `loan_end_date` set. `audit_entry.action` is one of `transfer`, `loan`, `release`, `loan_return`; `who` = `requested_by` (or `"system"` for `loan_return`); `when` = server time; `why` = `reason`.

`GET /audit/v1/roster/history/{player_id}?as_of=YYYY-MM-DD`

Response 200

```
{ "player_id": "p2", "as_of": "2027-01-05", "league_id": "lg-r1",
  "current_squad_id": "sq-a", "parent_squad_id": null,
  "history": [ { ...audit_entry (loan)... }, { ...audit_entry (loan_return)... } ] }
```

`history` lists entries oldest-first and is append-only. `as_of` is optional (default = today's server date); `current_squad_id` is the squad the player belongs to on that date (`null` if released). Unknown player → 404 `PLAYER_NOT_FOUND`.

5.12 Checkout and Inventory

`POST /audit/v1/checkout`

The grader first calls `/reset` with `stock` for the variant, then fires **20 simultaneous checkout requests** (Python threads released together by a barrier), each with a different `order_id`, buying the last unit of `var-jersey-001-M`. Exactly one must succeed.

```
{
  "order_id": "ord-001",
  "customer_id": "cust-001",
  "items": [ { "variant_id": "var-jersey-001-M", "quantity": 1 } ],
  "discount_code": null,
  "payment": { "token": "tok_success" }
}
```

- `order_id`: required, client-chosen, unique. Replay of an identical request → 200 with the stored result (no second reservation). Same id, different body → 409 `DUPLICATE_ORDER_ID`.
- `items`: 1–10 items, `quantity` integer 1..10, unique `variant_id`s. Unknown variant → 404 `VARIANT_NOT_FOUND`.
- `discount_code`: `null` or a code. Only `WELCOME10` is valid; unknown → 400 `INVALID_DISCOUNT_CODE`. `discount_minor = floor(subtotal_minor * 10 / 100)`.
- `payment.token` (mock gateway): `tok_success` (authorize + capture succeed); `tok_declined` (authorization fails); `tok_capture_fail` (authorization succeeds, capture fails).

Processing order and guarantees

1. Validate (400) → variants exist (404) → discount code (400).
2. Atomically reserve stock for all items. If any item lacks stock, reserve nothing and return 409 `OUT_OF_STOCK` with `details.variant_id`. Overselling or negative stock is a failure of NFR-3.
3. Call the mock gateway. `tok_declined` → release the reservation and return 402 `PAYMENT_DECLINED`. `tok_capture_fail` → void the authorization, release the reservation, return 402 `PAYMENT_CAPTURE_FAILED`. Released stock must be purchasable again.

4. On success: commit the reservation (stock decreases permanently), order status `paid`, generate the receipt.

Success response — 201 (200 on idempotent replay)

```
{
  "order_id": "ord-001", "status": "paid", "currency": "PKR",
  "items": [ { "variant_id": "var-jersey-001-M", "quantity": 1, "unit_price_minor": 450000 } ],
  "subtotal_minor": 450000, "discount_minor": 0, "total_minor": 450000,
  "payment": { "authorization_id": "auth_1", "capture_id": "cap_1", "state": "captured" },
  "receipt": {
    "payload": { "order_id": "ord-001", "customer_id": "cust-001",
                 "items": [ { "variant_id": "var-jersey-001-M", "quantity": 1, "unit_price_minor": 450000 } ],
                 "subtotal_minor": 450000, "discount_minor": 0, "total_minor": 450000, "currency": "PKR",
                 "paid_at": "2026-10-17T11:20:05Z" },
    "signature": "<hex>",
    "pdf_base64": "<base64 of the PDF file>"
  }
}
```

- `receipt.signature` = Section 5.3 with `id = order_id` and the canonical JSON of `receipt.payload` exactly as returned. The payload must be consistent with the order (totals add up).
- `receipt.pdf_base64` decodes to a valid PDF (starts with `%PDF`) generated server-side. Extracted text must contain the `order_id` and the total formatted as `PKR 4500.00` (major units, two decimals, no thousands separator).
- Out of stock → 409 with the error body of 5.2. Declined → 402.

`GET /audit/v1/inventory/{variant_id}`

```
{ "variant_id": "var-jersey-001-M", "stock": 0, "reserved": 0 }
```

`stock` = units still available for purchase. `reserved` = units held by in-flight checkouts; it must be 0 once all requests have returned. After the burst: exactly one 201, nineteen 409, `stock = 0` (never negative). The domain event for the order's status change must be emitted to your internal queue (not graded by script).

5.13 RBAC, Probe Routes and Security

`POST /audit/v1/rbac-matrix`

No body, no auth. **Response 200**

```
{
  "tokens": { "super_admin": "<jwt>", "league_organizer": "<jwt>", "club_manager": "<jwt>",
              "player": "<jwt>", "viewer": "<jwt>" },
  "context": { "tenant_id": "tenant-alpha", "foreign_tenant_id": "tenant-beta",
               "own_league_id": "league-alpha-1", "other_league_id": "league-alpha-2",
               "foreign_league_id": "league-beta-1",
               "own_club_id": "club-alpha-1", "other_club_id": "club-alpha-2", "foreign_club_id": "club-beta-1",
               "own_player_id": "player-alpha-1", "other_player_id": "player-alpha-2",
               "foreign_player_id": "player-beta-1" }
}
```

- Tokens are JWTs valid for at least 1 hour, sent as `Authorization: Bearer <jwt>`. All non-super-admin tokens belong to `tenant-alpha`. The league organizer owns only `league-alpha-1`; the club manager owns only `club-alpha-1`; the player is `player-alpha-1`; the viewer has read-only access inside tenant-alpha; the super admin is platform-wide. The ids are the seeded ids of Section 5.5.
- JWT claim names are your choice (the grader treats tokens as opaque). The signature must be verified; tokens with `alg: none`, a tampered signature, or an expired time must be rejected with 401.

Probe routes. They only authorize; they perform no mutation and do not validate the body. Allowed → 200 `{"authorized":true}`. Denied → 403 with the standard error body (`code = FORBIDDEN`). Missing/invalid token → 401 (`UNAUTHENTICATED`). The grader sends an empty body `{}` on POST/PATCH.

#	Probe route	Path variable
P1	`POST /audit/v1/rbac/tenants`	—
P2	`POST /audit/v1/rbac/leagues/{league_id}/fixtures`	league
P3	`POST /audit/v1/rbac/clubs/{club_id}/roster`	club
P4	`POST /audit/v1/rbac/clubs/{club_id}/products`	club
P5	`PATCH /audit/v1/rbac/players/{player_id}`	player
P6	`GET /audit/v1/rbac/clubs/{club_id}`	club
P7	`GET /audit/v1/rbac/platform/config`	—

Expected matrix (A = allowed → 200; D = denied → exactly 403)

Probe (target)	Super Admin	Organizer	Club Mgr	Player	Viewer
P1	A	D	D	D	D
P2 own_league	A	A	D	D	D
P2 other_league (same tenant)	A	D	D	D	D
P2 foreign_league	A	D	D	D	D
P3 / P4 own_club	A	D	A	D	D
P3 / P4 other_club (same tenant)	A	D	D	D	D
P3 / P4 foreign_club	A	D	D	D	D
P5 own_player	A	D	A (own club's players)	A (self only)	D
P5 other_player (other club)	A	D	D	D	D
P5 foreign_player	A	D	D	D	D
P6 own_club / other_club (tenant-alpha)	A	A	A	A	A
P6 foreign_club	A	D	D	D	D
P7	A	D	D	D	D

Cross-tenant access by a non-super-admin is always 403 (not 404). 404 is reserved for ids that do not exist at all. Authorization is evaluated before anything else about the request.

Additional security probes

Probe	Expected
P7 with no `Authorization` header	401 `UNAUTHENTICATED`
P7 with a super-admin token whose signature was altered, or with `alg: none`	401 `UNAUTHENTICATED`
Rate limit: 150 requests to `GET /api/v1/ping` within 5 s from one client	First request 200 `{"pong":true}`; at least 40 of the 150 return 429 (`RATE_LIMITED`) with a `Retry-After` header; 11 s later a request is 200 again
Input validation: `sport_id = x'; DROP TABLE sports;--`, or a `customer_id` containing quotes/`<script>`	Never a 5xx. Sport id → 400 `VALIDATION_ERROR`. Strings are stored/returned as plain data.
Body larger than 1 MB to any `/audit/v1/*` POST	413 `PAYLOAD_TOO_LARGE`

5.14 Offline Sync

`POST /audit/v1/offline-sync`

Replays the actions a mobile device queued while offline. The mobile app must queue actions in exactly this format (Section 8.4) and sync through this logic (the app may call this route or a production route backed by identical code).

```
{
  "device_id": "dev-1",
  "actions": [
    { "action_id": "a1", "type": "score_event", "queued_at": "2026-10-17T11:00:00Z", "base_version": 3,
      "payload": { "event_id": "evt-9", "match_id": "m1", "type": "score",
                   "payload": { "team_id": "t1", "event_type": "goal" } } },
    { "action_id": "a2", "type": "roster_transfer", "queued_at": "2026-10-17T11:01:00Z", "base_version": 1,
      "payload": { ...the same body as POST /audit/v1/roster/transfer... } }
  ]
}
```

- `actions`: 1–100 items, else 400 `VALIDATION_ERROR`. `type`: `score_event` (payload = the `/events` request body) or `roster_transfer` (payload = the `/roster/transfer` body).
- `base_version`: the version the device last saw. `score_event`: the match's version (number of events). `roster_transfer`: the number of audit entries of that player.
- **Processing order:** the server sorts actions by `queued_at` ascending (ties: array order) and processes them one by one, each against the state left by the previous one. Results are returned in request order.

Per-action result and conflict resolution (exact)

Situation (when this action is processed)	status	resolution	error_code
Rules allow it and `base_version == current version`	`applied`	`appended`	`null`
Rules allow it but `base_version < current version` (stale device)	`applied`	`rebased_append`	`null`
`(device_id, action_id)` already processed before, or the same `event_id` already exists with an identical payload	`duplicate`	`skipped_duplicate`	`null`
Any rule of 5.9 / 5.11 would reject the action (e.g. `MATCH_NOT_IN_PROGRESS`, `INVALID_COMPENSATION`, `NOT_IN_SQUAD`, `SQUAD_FULL`, `DUPLICATE_EVENT_ID`)	`rejected`	`server_wins`	that rule's code
`base_version > current version`	`rejected`	`server_wins`	`INVALID_BASE_VERSION`
Unknown id referenced (match, player, squad)	`rejected`	`server_wins`	the matching `*_NOT_FOUND`

Response 200

```
{ "device_id": "dev-1",
  "results": [ { "action_id": "a1", "status": "applied", "resolution": "rebased_append", "error_code": null,
                 "processed_order": 1, "server_version": 5, "signature": "<hex for score_event, else null>" },
               { "action_id": "a2", "status": "rejected", "resolution": "server_wins",
                 "error_code": "SQUAD_FULL", "processed_order": 2, "server_version": 1, "signature": null } ],
  "summary": { "applied": 1, "rejected": 1, "duplicate": 0 } }
```

`server_version` = the version of the affected match/player after processing (unchanged for rejected/duplicate). `signature` = the event signature of 5.9 for applied `score_event` actions. Re-sending the same request again must return `duplicate` for every action and change nothing. The whole request is always answered with HTTP 200 (conflicts are reported per action); only a malformed envelope is a 400.

6. Hidden Complexity Edge Cases

These are tested by the automated grading vectors and during the Top-8 Viva. The required behaviour is fully defined by the contracts above.

#	Edge case	Where / required behaviour
1	Round-robin with an odd number of teams and a mid-season team withdrawal	`/fixtures` round_robin: one bye per round; `withdrawals` + `withdrawal_policy` (`void_all` or `forfeit_remaining`) exactly as in 5.7-A.
2	Swiss rounds where no valid non-repeating pairing exists for some participants	`/fixtures` swiss: search globally (backtracking); if no rematch-free pairing exists for any bye candidate → 409 `NO_VALID_PAIRING` (5.7-B).
3	Double elimination with a bracket reset on the grand final	`/fixtures` double_elim: `GF2.conditional`; `GF2.required` becomes true only when the losers-bracket champion wins GF1 (5.7-D).
4	A three-way tie in the standings requiring the head-to-head mini-league tie-breaker	`/standings`: `resolve(group)` with h2h computed only among the tied teams, restart-on-split rule (5.10).
5	A match result corrected via compensating event after playoff qualification was already determined	`/events` compensation on a completed match; the next `/standings` call recomputes ranks, status, and h2h from scratch; original events stay unchanged in `/score` (5.9, 5.10).
6	A player transferred on the last day of the transfer window while having an in-progress match	`/roster/transfer`: accepted with `state:"deferred_until_match_end"` and `blocking_match_id` (5.11).

7. Approved Open-Source Stack

Category	Approved open-source software
Database, cache & storage	PostgreSQL 16, Redis 7, MinIO
Auth & message queues	Custom JWT / Keycloak / Ory Kratos; RabbitMQ / Redis Streams / NATS / BullMQ / Celery
PDF generation	WeasyPrint, wkhtmltopdf, Puppeteer
Testing & load probes	pytest, JUnit, TestNG, grafana/k6

Third-party fixture-generation libraries and managed BaaS products are prohibited. Web frameworks, ORMs, JWT libraries, and PDF libraries are fine.

8. Mobile Application Requirements and Judging

8.1 Platform and Delivery

- **Android (recommended):** provide `mobile/release/app-release.apk`, installable on Android 10 (API 29) or newer. **iOS teams** must bring their own device or simulator with the app already installed; the judging slot is the same.
- The app talks to the backend on the team laptop (docker compose running). The server address must be changeable inside the app (settings screen or build config): the emulator uses `10.0.2.2:8080`, a phone uses the laptop's LAN IP.
- Secure token storage is mandatory (Android Keystore / iOS Keychain).

8.2 Required Screens

Screen	Must show
Login	Email + password; role-aware home after login.
Fixtures	List of matches with status and time; tap to open a match.
Match detail	Live score (updates in real time), event timeline, pinned product showcase.
Standings	Points table for a league with qualification status.
Club profile	Club info, roster, staff, home venue.
Store	Catalogue, variant picker, cart, checkout with the mock gateway, receipt view/download.
Roster management	For Club Manager: add / transfer / loan / release a player.
Scoring entry	For League Organizer: append a score event and a compensating event.
Sync status	Always-visible online / offline / syncing indicator and the pending-action queue.

8.3 Demo Accounts

Must exist after every startup and every `/reset`.

Role	Email	Password
Platform Super-Admin	`admin@nascon.test`	`Nascon@26`
League Organizer (owns `league-alpha-1`)	`organizer@nascon.test`	`Nascon@26`
Club Manager (owns `club-alpha-1`)	`manager@nascon.test`	`Nascon@26`
Player (`player-alpha-1`)	`player@nascon.test`	`Nascon@26`
General Viewer	`viewer@nascon.test`	`Nascon@26`

8.4 Offline Tolerance (NFR-4) — What the Desk Check Does

1. With the backend reachable, the judge opens Fixtures, Standings, and a Club profile. The app must cache them in a local database (SQLite / Room / Hive).
2. The judge switches the phone to airplane mode. Cached screens must still open and an "Offline" indicator must show.
3. While offline the judge performs one scoring action (as Organizer) and one roster action (as Manager). Each must be stored in a persistent local queue, appear as "pending" in the UI, and use the action format of 5.14.
4. The judge force-closes and reopens the app while still offline. The pending actions must still be there.
5. The judge turns the network on. The app must sync automatically (or via an obvious Sync button), show

per-action results (applied / rejected + reason), and clear the queue. Rejected actions must be explained in plain language.

8.5 Judging Procedure

- From 15:15 PKT judges visit each team at its venue desk. Each team gets one fixed **10-minute slot**. The team must have the backend running and the app installed.
- All judges use the same three tasks (Section 8.6). Judges do not help the team during the slot.
- A team that is not ready at its slot is moved to the end of the queue once; if still not ready, the desk-check parts (offline and UI/HCI) score 0.

8.6 What the UI/HCI Judges Look At

Judges run the same three tasks on every team's app: **(A)** find a league's standings and open one match; **(B)** buy a merchandise item with the mock payment and open the receipt; **(C)** as Club Manager, perform a roster action — then repeat one action in airplane mode and sync. They judge five areas:

Area	What we check
1. Navigation & ease of use	A new user can find Fixtures, Standings, Store, and Profile without help. Common tasks take few taps. Back/undo always works. No dead ends.
2. Visual consistency & hierarchy	Same colors, fonts, spacing, and button styles on every screen. The most important things (score, Buy button) stand out. The tenant's logo and colors are applied.
3. Feedback & error handling	The app always tells the user what is happening: loading indicators, success messages, clear plain-language errors (never raw codes like "Error 500"), visible Online / Offline / Syncing status, confirmation before risky actions.
4. Readability & accessibility	Text is large enough, contrast is good, tap targets are comfortably big (about 48 dp), icons have labels or descriptions, meaning is never shown by color alone.
5. Right screen for the right role	Each role sees only what it can use. Empty lists show a helpful message ("No matches yet"). Nothing breaks or overflows on a small phone (360×640 dp) or after rotating.

9. Phase I Scoring and Rubric (50 Points)

9.1 Point Groups

Phase I graded component	Max	Verification method
Backend Platform — six areas, listed below	36	Python grading scripts on the grader machine
Offline Mobile Persistence & Sync Queue	4	Python script (`/offline-sync`) + on-device desk check (8.4)
Mobile UI & HCI Principles — five areas in 8.6	10	Judge panel, same three tasks for every team
Total Phase I score (permanently locked at 15:00 PKT)	**50**	

The six backend areas (together worth 36 points):

- Fixture algorithms ×4 (round-robin, Swiss, single elimination, double elimination) against hidden vectors
- Event-sourced scoring engine (mystery-sport configuration, compensating-event rollback)
- Standings engine (3-way H2H mini-league, differentials, clinched / eliminated status)
- Store concurrency (20 simultaneous buyers on the last unit of stock, PDF receipt)
- Club, roster, transfer-window, and loan rules (hidden conflict and audit-trail vectors)
- RBAC 5-role negative matrix, tenant isolation, and API security (boundary probes)

9.2 Scoring Function

For every backend area worth P points, tested against $|V| \geq 40$ hidden vectors of which n pass:

$$S \;=\; 0.9\,P\,\frac{n}{|V|} \;+\; 0.1\,P\,\big[\text{unit-test coverage of that area} \geq 85\%\big]$$

- A vector passes only if *every* assertion in it holds (status code, error code, every compared field, signatures).

There is no partial credit inside one vector; partial credit comes from passing more vectors.

- The offline-sync script portion uses the same proportional rule (no coverage bonus). UI/HCI is scored by judges.
- Hidden vectors include seeded sports and newly created mystery sports, odd/even and non-power-of-two sizes, and every error code listed in Appendix B.

9.3 Coverage Bonus File

Commit `COVERAGE.json` at the repository root with line-coverage percentages for the six backend areas:

```
{ "fixtures": 91.2, "scoring": 88.0, "standings": 86.5, "store": 90.1, "roster": 87.3, "rbac": 85.0 }
```

The grader reads this file. Judges may run `./run_tests.sh` for any team; if the real figure is more than 2 percentage points below the declared one, that area's bonus is voided.

9.4 Penalties

Floored at 0; applied to the total of 50.

Penalty	Points	Definition
Non-reproducible Docker stack	−10	`docker compose up -d --build` fails, exceeds the time limits of 5.2, or needs manual steps (after one automatic retry).
Hard-coded sport logic	−8	Any branch on a specific sport or event name (e.g. "football", "goal") in the domain logic, found by code scan or revealed by the mystery-sport vectors.
Prohibited managed BaaS	−10	Any proprietary single-vendor backend service used for a graded feature.
Missed GitHub push checkpoint	−2 each	A window of Section 1.3 with no pushed commit (maximum −8).

9.5 Final Ranking and Disputes

- Ranking uses the final score. Ties are broken by: (1) higher backend score, (2) fewer penalty points, (3) earlier timestamp of the final pushed commit.
- The script result is final. The organizers re-run the grader once only if a failure is clearly caused by the organizers' environment (not by the team's stack).
- Questions about the interpretation of this contract must be raised **in writing before 13:00 PKT**; the organizers' written answer applies to all teams. Score appeals must be filed within 30 minutes of the results being shown; the organizers' decision is final.

A. Reference Code and Test Vectors

```
import hmac, hashlib, json

def canonical(obj):                        # Section 5.3
    return json.dumps(obj, sort_keys=True, separators=(",", ":"), ensure_ascii=False)

def sign(salt: str, id_: str, payload_obj) -> str:
    msg = (id_ + "|" + canonical(payload_obj)).encode("utf-8")
    return hmac.new(salt.encode("utf-8"), msg, hashlib.sha256).hexdigest()

def lots_key(seed: int, league_id: str, team_id: str) -> str:   # Section 5.10, draw_lots
    return hashlib.sha256(f"{seed}|{league_id}|{team_id}".encode("utf-8")).hexdigest()
```

Test vector 1 (signature). Salt `demo-salt`, id `evt-1`, payload `{"team_id":"t1","event_type":"goal"}`:

```
canonical payload = {"event_type":"goal","team_id":"t1"}
signature = 972a8b558690b19121cc5bea84978f85e8eb15a3735018db36616bab1d7ec501
```

Test vector 2 (drawing of lots). Seed 42, league `lg-1`; teams are ordered ascending by key, first = best:

```
t1 -> 0613316991ce0ac31e88d6003e3ff4d5e21bb072ac5f81b42c0cdc6598453293
t2 -> e3e3a14e22727f936e1f288cf5d02a8a9362b1da13c5432725fc10b2c7955dd9
t3 -> c8f28e3c2f1eb6c4f636b4f57050e3362a2b39c07fda9b61742a26dd20397028
```

B. Error Code Catalogue

HTTP	Code	When
400	`VALIDATION_ERROR`	Schema/type/range problem (`details.fields`), bad dates, bad limits.
400	`UNKNOWN_EVENT_TYPE`	`score event_type` not defined in the sport.
400	`INVALID_DISCOUNT_CODE`	Unknown discount code.
401	`UNAUTHENTICATED`	Missing / invalid / tampered / expired / `alg:none` token.
402	`PAYMENT_DECLINED`, `PAYMENT_CAPTURE_FAILED`	Mock gateway failure.
403	`FORBIDDEN`	Role or tenant/ownership boundary.
404	`SPORT_NOT_FOUND`, `MATCH_NOT_FOUND`, `LEAGUE_NOT_FOUND`, `EVENT_NOT_FOUND`, `PLAYER_NOT_FOUND`, `SQUAD_NOT_FOUND`, `VARIANT_NOT_FOUND`	Referenced id does not exist.
409	`SPORT_EXISTS`, `LEAGUE_EXISTS`, `MATCH_EXISTS`	Id already used.
409	`DUPLICATE_EVENT_ID`, `DUPLICATE_ORDER_ID`	Same id re-used with a different body.
409	`ILLEGAL_TRANSITION`, `MATCH_NOT_IN_PROGRESS`, `TIE_NOT_ALLOWED`	Match state-machine violations.
409	`INNINGS_COMPLETE`, `SERIES_ALREADY_DECIDED`	Config-driven scoring limits.
409	`INVALID_COMPENSATION`	Target already compensated, wrong type, or other match.
409	`NO_VALID_PAIRING`	Swiss: no rematch-free pairing exists.
409	`OUT_OF_STOCK`	Not enough stock.
409	`CROSS_TENANT_TRANSFER`, `NOT_IN_SQUAD`, `TRANSFER_WINDOW_CLOSED`, `PLAYER_ON_LOAN`, `SQUAD_FULL`, `MIN_POSITION_VIOLATION`	Roster rules (checked in the order of 5.11).
—	`INVALID_BASE_VERSION`	Only inside an offline-sync per-action result (5.14).
413	`PAYLOAD_TOO_LARGE`	Body over 1 MB.
429	`RATE_LIMITED`	`/api/v1/*` limit exceeded.

C. Pre-Submission Checklist

- `DAY1_SALT=<salt> docker compose up -d --build` works on a clean machine; `/health` is 200 in under 2 minutes.
- `POST /audit/v1/reset` twice in a row leaves identical data; the 5 demo accounts and all seed ids exist.
- Every response matches the field names in Section 5 (compare your JSON with the examples character by character).
- Signatures verified against Appendix A test vector 1 using your own salt logic.
- 20 parallel checkouts on stock 1 → one 201, nineteen 409, stock 0, reserved 0.
- `/audit/v1/*` is not rate limited; `/api/v1/ping` is.
- `COVERAGE.json`, `TRACEABILITY.md`, `README.md`, `.env.example`, and `mobile/release/app-release.apk` are committed and pushed.
- Last commit pushed before 15:05:00 PKT.

If your code works on the first try, check again. It's probably the wrong problem.

Understanding The Problem Is Also Part Of The Problem.